

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture No. 7



Re Cap - Questions regarding Regular Languages

Natural Questions:

1. Is *Union/Intersection/Complement* of regular languages regular? **Union Yes**, just by the definition of regular expression
2. Is *Concatenation/Kleene Star* of regular languages regular? **Yes**, just by the definition of regular expression
3. *How many* regular languages are there?
Or *Are all languages regular*?
4. *If not*, which language is not regular and how to show that language is not regular?

Re Cap - Questions regarding Regular Languages

Answer to Question 3

Remember

Theorem

The set $\mathcal{R}$ of **regular expressions** over an alphabet Σ is **countably infinite**

A language $L \subseteq \Sigma^*$ is **regular** if and only if

L is represented by a **regular expression**

Therefore, the set of regular languages on Σ is countably infinite

Whereas set of all languages on Σ is uncountably infinite.

So **uncountably infinite languages are not regular.**

Re Cap - An example: Controlling a toll gate

We consider the problem of designing a “*computer*” that controls a toll gate.

When a car arrives at the toll gate, the gate is closed.

The gate opens as soon as the driver has payed 25 cents.

We assume that we have only three coin denominations: 5, 10, and 25 cents.

For simplicity, we also assume that no excess change is returned.

Controlling a toll gate

After having arrived at the toll gate, the driver inserts a sequence of coins into the machine.

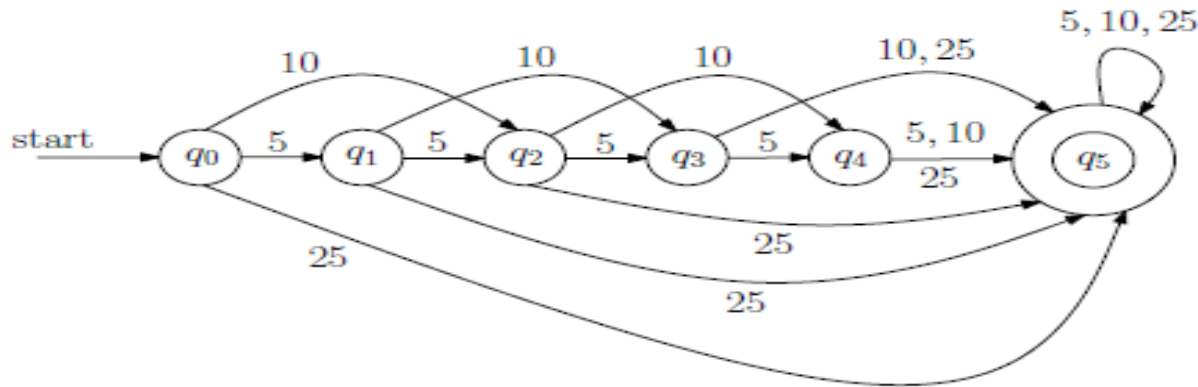
At any moment, the machine has to decide whether or not to open the gate, i.e., whether or not the driver has paid 25 cents (or more).

In order to decide this, the machine is in one of the following *six states*, at any moment during the process:

- The machine is in state (*initial state*) q_0 , if it has not collected any money yet.
- The machine is in state q_1 , if it has collected exactly 5 cents & so on

Controlling a toll gate

We can represent *pictorially* the behavior of the machine for all possible sequences of coins.



State q_5 is represented by *two circles*, because it is a special state: As soon as the machine reaches this state, the gate opens.

State q_0 has an incoming arrow, indicating it is the *start state*

Controlling a toll gate

Observe that the machine (or computer) only has to remember which state it is in at any given time.

Thus, it needs only a very small amount of memory

Coffee/Tea vending machine

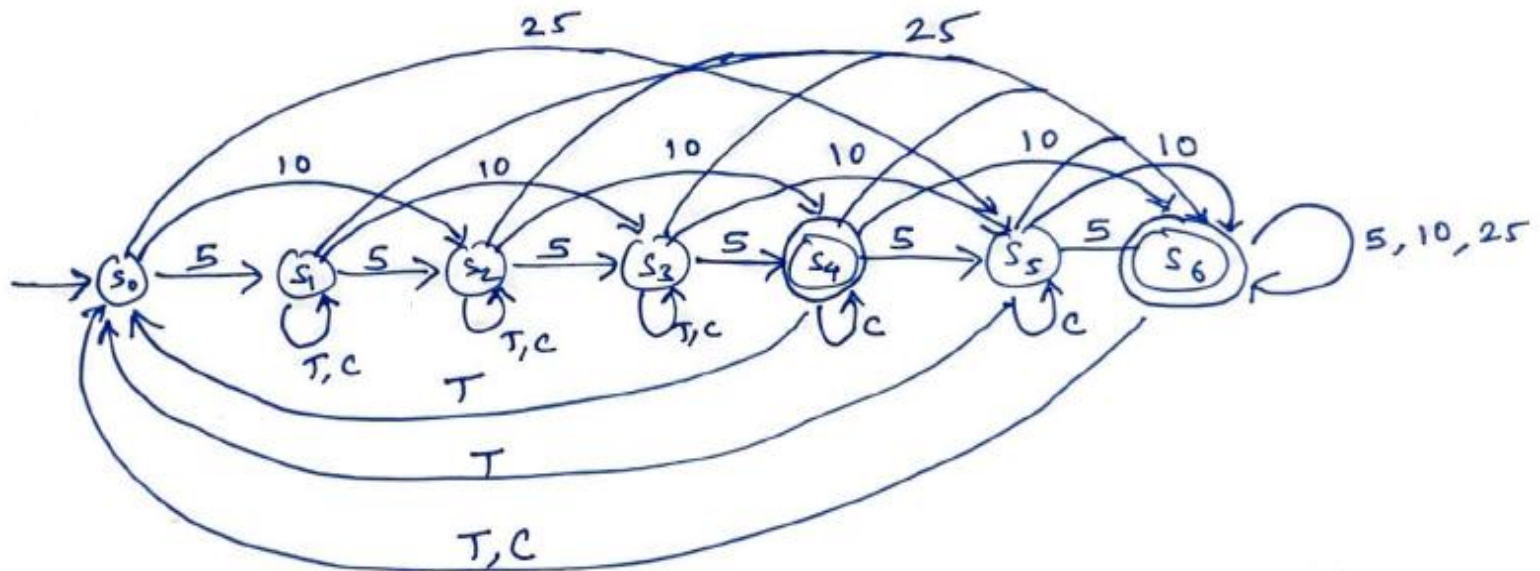
We will show how a **Coffee/Tea vending machine** can be modeled.

Vending machine accepts 5 cents, 10 cents and 25 cents.
When a total of 20 cents or more has been deposited,
the customer can push a tea button and receive tea
When a total of 30 cents or more has been deposited
the customer can push a coffee button and receive
coffee

We can describe how the machine works by specifying its states, how it changes states when input is received, and the output that is produced for every combination of input and current state

Coffee/Tea vending machine

Note: There are two '*accepting*' states



Deterministic Finite Automaton

Operations of Finite State Automaton

Strings are fed into the device by means of an *input tape*.

The *input tape* is divided into squares with one symbol inscribed in each square.

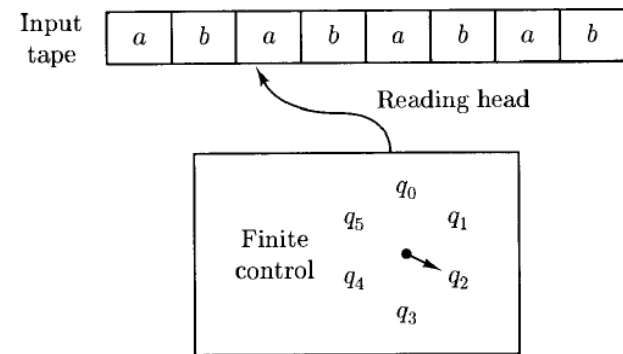
The main part of the machine called *finite control*; at any point it is in one of the *finite* number of *states*

Finite Control can read the symbol written at any position of the input tape by a movable *reading head*.

Deterministic Finite Automaton

Initially, *reading head* is pointing at the left most square of the tape and the finite control is in a designated *initial state*.

At regular intervals, machine reads one symbol from the input tape & then enters a new state *that depends only on the current state & the symbol just read*.



And so we call this device a *deterministic finite automaton*.

After reading an input symbol, the reading head moves one square to the right on the input tape.

This process is *repeated* again & again till all symbols of input tape are read.

Deterministic Finite Automaton (DFA)

Formal Definition

A *deterministic finite automaton* is a 5-tuple

$M = (K, \Sigma, \delta, s, F)$, where

K is a finite set, whose elements are called *states*

Σ is a finite set, called the *alphabet* set

$s \in K$ is the *initial state*

$F \subseteq K$ is set of *final/accept* states

$\delta : K \times \Sigma \rightarrow K$ is a function called the *transition function*

DFA for the “toll gate”

The *‘computer’* we defined for the *toll gate* is a DFA.

$$K = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

$$\Sigma = \{5, 10, 25\}$$

$$s = q_0 \text{ \& } F = \{q_5\}$$

And δ is give by

	5	10	25
q_0	q_1	q_2	q_5
q_1	q_2	q_3	q_5
q_2	q_3	q_4	q_5
q_3	q_4	q_5	q_5
q_4	q_5	q_5	q_5
q_5	q_5	q_5	q_5

DFA - Example

Example: Let $M = (K, \Sigma, \delta, s, F)$, where

$$K = \{q_0, q_1\}$$

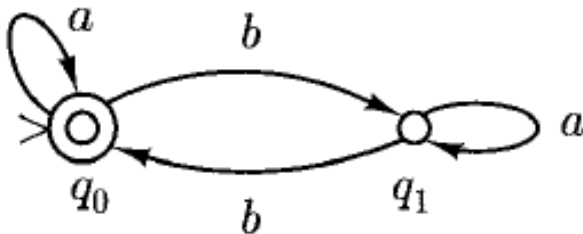
$$\Sigma = \{a, b\}$$

$$s = q_0 \text{ \& } F = \{q_0\}$$

And δ is give by

q	σ	$\delta(q, \sigma)$
q_0	a	q_0
q_0	b	q_1
q_1	a	q_1
q_1	b	q_0

Graphical representation (called *state diagram*)



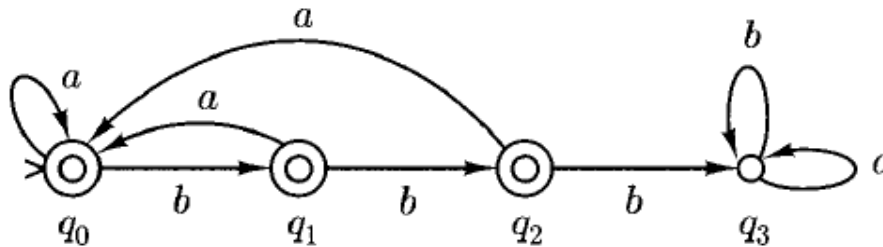
Which strings are '*accepted*' by M?

Strings in $\{a, b\}^*$ that have *even number of b's*

DFA - Example

Example:

$L = \{w \in \{a, b\}^* : w \text{ does not contain three consecutive } b's\}$



and δ is given by the following table.

We let $M = (K, \Sigma, \delta, s, F)$, where

$$K = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\},$$

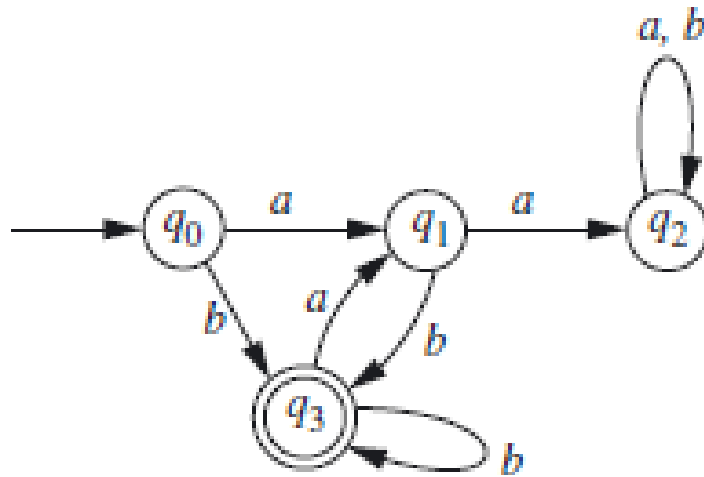
$$s = q_0,$$

$$F = \{q_0, q_1, q_2\},$$

q	σ	$\delta(q, \sigma)$
q_0	a	q_0
q_0	b	q_1
q_1	a	q_0
q_1	b	q_2
q_2	a	q_0
q_2	b	q_3
q_3	a	q_3
q_3	b	q_3

DFA

$L = \{w \in \{a, b\}^* \mid w \text{ ends with } b \text{ and does not contain the substring } aa\}$



We see that every DFA *‘generates’* a language.

DFA Exercise

Ex $L = \{w : \{0,1\}^* : w \text{ has a } 1 \text{ in the third position from the right}\}$

How does the automaton “*know*” that it has reached the third symbol from the right?

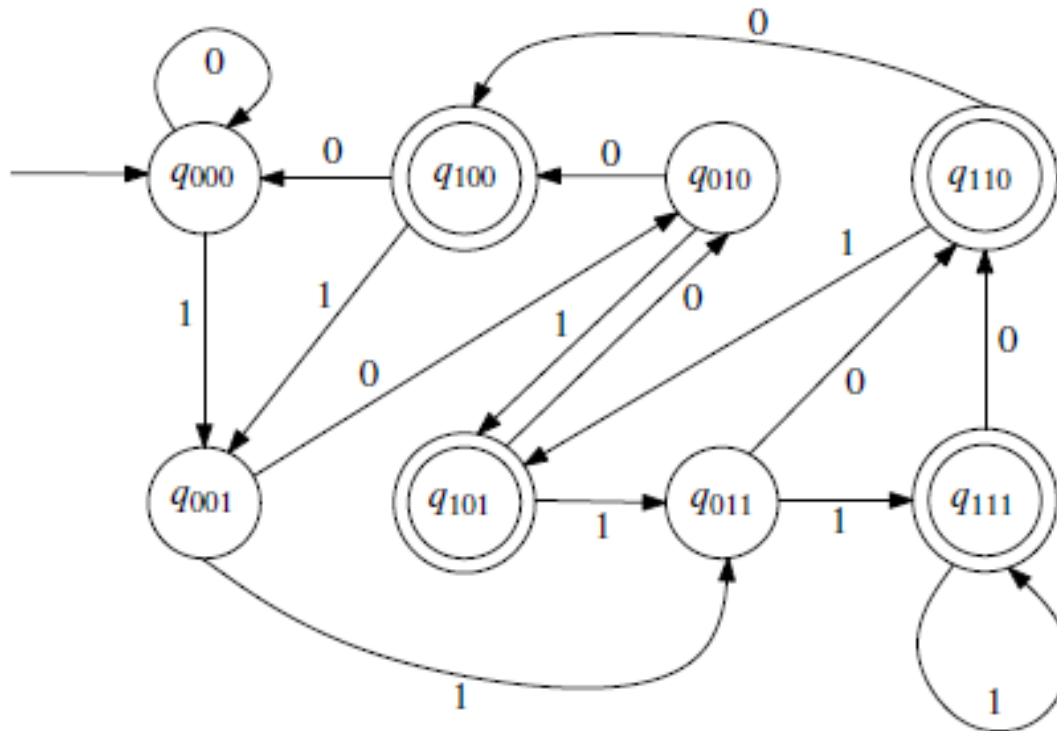
The *main idea* is to remember the last three symbols that have been read.

If the automaton is in state q_{ijk} then the following hold:

- If M has read at least three symbols, then the three most recently read symbols are ijk .
- If M has read only two symbols, then these two symbols are jk ; moreover, $i = 0$.
- If M has read only one symbol, then this symbol is k ; moreover, $i = j = 0$.
- If M has not read any symbol, then $i = j = k = 0$.

DFA Exercise

Thus, the finite automaton has eight states q_{ijk} , where i, j , and k range over $\{0, 1\}$.



‘Computation’

Next we understand the notion of *‘computation’* by an DFA

Note

DFA is not allowed to move its *reading head* back into the part of input string that has been already read.

So the portion of the string to the left of the *reading head* does not influence the future operations of the machine.

A *‘configuration’* of the DFA is determined by the current state & the unread part of the input string.

Computation

Configuration of DFA $(K, \Sigma, \delta, s, F)$ is an element of $K \times \Sigma^*$

In this figure the *configuration* is $(q_2, ababab)$

Computation is DFA moving

from a configuration to another configuration in one step.

Computation is an binary relation on two configurations of the DFA

